

# Knowledge Tracing with Vision-Language Models

## Two Approaches to Student Performance Prediction

June 29, 2026

### Problem Statement

Knowledge Tracing (KT) is the task of predicting whether a student will answer the next exercise correctly, given their history of past attempts. This work explores two experimental pipelines trained and evaluated on the **MIAAM dataset**, which contains 38,519 students with an average of 168 exercises each. Both pipelines leverage **SmolVLM-256M-Instruct** as the multimodal backbone and share a common data representation: exercises are described by text and a screenshot, while attempts are described by text (answer given, duration, timestamp, and correctness label).

The first experiment is intended as a **baseline**: feeding raw multimodal content directly to a VLM is the most straightforward approach to the problem, and it helps characterise the limitations of this strategy on the MIAAM dataset. The second experiment proposes an alternative design in which the VLM is used to encode each exercise and each attempt *independently* into a fixed-size vector; these representations are then consumed by a Transformer architecture, which can reason over much longer student histories.

### Experiment 1 — Raw Multimodal Input

The first approach feeds *raw* multimodal content directly to the VLM at training time. For a student with history  $t$ , the model receives a concatenated sequence of exercise screenshots and text, together with text descriptions of past attempts.

**Objective.** Establish a baseline by fine-tuning a VLM either end-to-end or partially, directly on the raw multimodal input.

Two training regimes are supported:

- **Probe** — the VLM backbone is fully frozen; only a linear classification head appended to the pooled hidden state is trained. This is fast and parameter-efficient.
- **LoRA fine-tuning** — lightweight LoRA adapters are injected into the attention and MLP projections of the backbone and trained end-to-end alongside the head, at a lower learning rate than the probe.

**Key limitations.** The 256M-parameter VLM has a small context window, which restricts each input to only 4–5 past attempts regardless of the student’s actual history length. Since students average 168 exercises, the vast majority of each student’s learning trajectory is discarded at every training step, inherently limiting what the model can infer about long-term knowledge state.

A second limitation is **training speed**. Even in probe mode, where the backbone is frozen and no gradients flow through it, a full VLM forward pass must run on every batch to produce the hidden states consumed by the classification head. Processing raw screenshots through the vision encoder at every step is expensive, and this cost is paid repeatedly across all epochs. By contrast, Experiment 2 pays the VLM inference cost only once during preprocessing: embeddings are cached to disk and the training loop never touches the VLM again, making each training step orders of magnitude cheaper.

### Experiment 2 — Pre-computed Embeddings

The second approach decouples the VLM from the training loop entirely. Each exercise and each attempt is encoded *once* by a frozen VLM into a 576-dimensional embedding vector. A lightweight Transformer then learns to reason over sequences of these cached embeddings.

**Objectives.** (1) Validate the advantage of pre-computing exercise and attempt embeddings from a computational perspective, expecting faster training that can run on smaller GPUs. (2) Validate the advantage in terms of prediction performances of feeding the model a longer sequence of student history.

**Input sequence.** For a student who has completed  $t$  exercises, the model receives an interleaved sequence of  $2t + 1$  vectors:

$$[e_0, a_0, e_1, a_1, \dots, e_{t-1}, a_{t-1}, e_t]$$

where  $e_i = \text{VLM}(\text{exercise}_i)$  and  $a_i = \text{VLM}(\text{attempt}_i)$ . The final token  $e_t$  has no corresponding attempt yet; the prediction head reads the Transformer’s hidden state at that position and produces a scalar logit for binary classification.

**Architecture.** The **KTTransformer** consists of (i) a linear projection from the 576-d input space to the model’s internal dimension  $d_{\text{model}}$ , (ii) learnable positional embeddings, (iii) a stack of standard Transformer encoder layers (bidirectional attention, no causal mask), and (iv) a two-layer MLP head. Sequences are zero-padded to the batch maximum and a boolean attention mask prevents the model from attending to padding tokens.

**Key advantage.** Because embeddings are pre-computed and compact, the Transformer can process histories of up to **512 past exercises**, covering most students in full rather than truncating their history.

## Results

No complete results was found for Experiment 1. A single training epoch on the available GPU takes 3–4 hours due to the cost of running the full VLM forward pass at every step, making a full training run impractical within the available time budget.

The embedded approach reaches **83% validation accuracy** after a short training session, with no hyperparameter tuning. This is a strong baseline given the simplicity of the encoder (256M parameters) and the absence of any hand-crafted features.

## Conclusion

Experiment 1 lets the VLM jointly encode each exercise and reason over the full history end-to-end, so it can interpret an exercise in light of what just happened (e.g. right after a failure vs. a success), at the cost of higher capacity demands and the risk of entangling exercise content with spurious features of any one student’s trajectory. Experiment 2 factorizes the problem — a frozen encoder produces consistent, context-free exercise embeddings that a dedicated downstream transformer contextualizes over the sequence — which is more parameter-efficient and avoids baking history-specific noise into the content representation, but loses the ability to fuse context at encoding time if the embedding discards the relevant detail.

Regarding Experiment 1, it would still be interesting to run a complete training on this setup, potentially using a more powerful model with a larger context window to better capture student history.

Experiment 2 appears very promising, especially when one of the main desiderata of the project is to operate with small VLMs. It offers clear advantages in training speed and in the ability to feed the model a much longer student history. The natural next step here is to try a slightly larger backbone to produce richer embeddings, while retaining the decoupled, computationally efficient design.

## Use of GenAI

I usually write code using VSCode with the Claude Code plugin installed as an AI code assistant. To develop this project I have used the AI assistant as I would more or less all the time. I would ask the AI assistant to write specific chunks of code (usually 100-200 lines of code). I would double-check and test and ask to rewrite some pieces. I would iterate in this way until the code is doing what I wish, to then move on to writing new chunks of code.

In this project almost all the code has been written in the previous way. To be more accurate this is how I used the AI assistant.

In the notebooks for data preparation I did not use much the AI assistant except for writing small chunks of code (around 20 lines). Once the data preprocessing was over, I started by asking the assistant to generate a PyTorch dataset class that could load data from the dataframe I created. I was giving to the assistant the header and first rows of the sequence of attempts and the exercises created in the notebook (create\_per\_student\_sequence.ipynb). I was describing the task to the assistant in this way:

**Build a PyTorch Dataset (raw\_data\_experiments)** *Can you write a PyTorch Dataset class for knowledge tracing that produces VLM-ready inputs?*

*sequences.parquet: per-student chronological attempt history, each attempt containing exercise\_id, data\_correct,...*

*exercises.parquet: per-exercise metadata.*

*A screenshot PNG per exercise at DATASET/data/screenshots/compressed/exercise-id.png. To obtain a training you should: Pick a random split point  $t$  in a student's history. The input should be the history up to time  $t$  plus the text description and screenshot of exercise  $t+1$ . The target should be the success/failure of exercise  $t+1$ . Target should be used as input to a very small Hugging Face VLM (SmolVLM-256M-Instruct)*

After this I had a few iterations to ask to have parameters to cut the student history and I reviewed the outcome of the code (see notebook tests/data\_processing\_test.py) I did something similar for the second experiment.

**Build a PyTorch Dataset (embedded\_data\_experiments)** *I would like some help in writing a pytorch dataset class. The class will receive as input a pandas file with 2 columns: 1) Student history 2) Student scores The student history column contains embeddings of the exercises performed and info around the attempt. The exercises and the attempts descriptions are represented in vectors of the same size  $D$ , in a list:  $history = [exercise\_1, attempt\_1, \dots, exercise\_n, attempt\_n]$  The scores are true or false:  $scores = [score\_1, \dots, score\_n]$  I would like the `get.item` to randomly select a point in history ( $t$ ), return all the exercises and the attempts done before and the score at time  $t$ .  $history = [exercise\_1, attempt\_1, \dots, exercise\_t]$   $score = score\_t$*

Going back to the experiments in the raw\_data\_experiments folder, after I had a good working dataset class, I asked the AI assistant to build a model.

**Build a Model** *Can you write me a PyTorch model that adds to the last layer of the VLM HuggingFaceTB/SmolVLM-256M-Instruct a linear layer? I would like you to use the transformer class to load the VLM. The linear layer should receive as input the pooling of the last layer of the VLM, the pooling should return the embedding of the last token.*

After validating that the code was respecting my requirements, I noticed that the VLM was not frozen when the model class was initialized, so the code was not setting `required_grad = False` for the params of the VLM. So I asked the assistant to fix this. After few iterations I obtained the class KTProbe in raw\_data\_experiments/models.py

After that I gave the AI assistant the KTProbe class and asked to modify the code so that I could fine-tune the entire VLM with LoRA. I obtained in this way the class KTLORA.

**Modify the Model to Use LoRA** *This is the code I am using for training a last layer on top of a VLM. I would like to test another architecture where the entire VLM is fine-tuned using LoRA.*

Finally I asked to write a simple training script using only a training and validation loop, without saving the model params. I specified that the training script should use the classes defined before for loading data and for the model architecture. I then asked to: add a progress bar showing the training status, and add a config file using YAML.

For the architecture in the second experiment (embedded\_data\_experiments) I was asking the AI assistant:

**Write a smile transformer class** *Write a standard transformer architecture. The input is a padded sequence of per-student history embeddings of shape  $B \times T \times D$ , interleaved exercise and interaction embedding vectors of dimension  $D$ , together with a  $B \times T$  boolean attention mask marking real tokens versus padding. The output is a  $B$  tensor of logits giving, per student, the predicted probability of solving the next, target, exercise.*